

CRAgent_TS 提问归纳与改进建议

基于 CRAgent_TS 项目 50+ 次对话、约 200+ 条提问整理 · 2026-06-01

一、你在 CRAgent_TS 里主要问了什么

1. 项目从零到可用（早期，最大的一块）

主题	典型提问
技术迁移	参考 CRAgent, 用 Electron 26 重写, 兼容 macOS 10.14
功能对齐	文件读写、bash、skill、AGENTS.md/记忆系统、sub-agent、todo、fallback
打包发布	DMG 签名、Windows 安装包、iPad 包、推送到 GitHub、写 README

这是「建房子」阶段：目标大、步骤多、需要持续跟进（「继续」「卡在哪里？」）。

2. UI / 视觉还原 Cursor（占比最高）

你大量时间在微调界面，集中在：

- **Composer 输入区**：高度、阴影、虚化、行数自适应、拖图、回车发送、排队、中断按钮、context 圆环
- **历史会话**：Thinking 折叠、runId 分组、复制按钮、模型名显示、背景色、滚动条位置
- **Session 导航**：新建会话、设置按钮、动画、图标
- **Settings 页**：Agent/Context/Models 卡片化、Provider 增删改、模型列表过滤
- **标题栏 / App Icon**：样式、拖拽、tooltip、图标颜色

特点：精确到像素（4px、8px、10px）、对照截图/录屏、指定 CSS 类名（如 composer-toolbar、composer-send）。

3. Bug 修复

问题	你的表述方式
confirmToolExecution is not a function	直接贴报错
skill 无法调用	与上面 8 项需求一起提
crypto is not defined	运行时错误
发图后发送按钮不可点	测试反馈
切换 session 时输入框不可用	行为描述
import 缺失、窗口空白	贴终端日志

4. 架构 / 原理类（质量较高）

- 上下文超过阈值后怎么处理？有几种压缩方式？
- 与 Claude Code 压缩方式对比 → 按对比结果实现 → 继续优化 → Settings 开关
- 传给 LLM 的 messages 结构是什么？
- /clear 语义：不清 UI，只清 LLM 上下文

这类提问 **先理解再改**，迭代路径清晰，效果通常更好。

5. 运维 / 工具链

- 更新到 GitHub、配置 GH_TOKEN、做成 skill
- DMG 签名证书指定
- iPad 编译与测试

6. 你的提问风格特征

做得好的：

- 给出 CSS 类名 / 文件路径，定位很准
- 用截图、录屏、参考 JSON 说明期望效果
- 偶尔明确范围边界（「只改 Agent/Context 页，其他地方不动」「遵守卡帕奇合约，别乱改」）
- 对标 Cursor / Claude Code，有明确参照物

容易拖慢效率的：

- 一次提 8~10 条不同层级需求（修 bug + 新功能 + UI 微调混在一起）

- 极短追问：「没看到效果」「修复好了吗」「继续」「去掉这个」——缺少复现步骤
- 像素微调多轮：「往左 4px → 往右 2px → 再加深一点」
- 同需求开新对话重复提（如去掉圆角、iPad 包、Agent 卡片化）
- 截图只说「红色框/箭头」，未说明当前行为 vs 期望行为
- 需求中途变更（如 auth 菜单先要有图标，后来又「去掉图标」）

二、如何提问效果更好

原则：一条消息 = 一个可验证结果

把需求拆成：

目标 → 位置 → 期望行为 → 验收标准 → 不要动什么

弱：

复制按钮应该靠右对齐

强：

用户消息气泡里的复制按钮应靠右对齐（与 Cursor 一致）。

文件：ChatView.jsx / chat.css

验收：短消息、长消息、带图片消息三种情况都靠右

不要改：删除按钮、时间戳区域

按任务类型选模板

A. Bug 修复

【现象】发送消息后报错 `confirmToolExecution is not a function`

【复现】1. `npm run dev` 2. 发送任意消息 3. 触发工具调用

【期望】正常弹出确认并执行

【日志】（贴终端/控制台）

【范围】只修这个错误，不改 UI

B. 新功能

【功能】消息排队：发送中可继续输入，回车入队

【参考】截图 + `/Users/airdroid/Downloads/xxx.json`

【优先级】P0 排队逻辑 P1 队列面板 UI P2 中断按钮样式

【不要】本次不做 todo 自动运行

大功能分 2~3 个对话做，比一次 8 条更稳。

C. UI 微调

- 【元素】 composer-auth-wrap 与 composer-send 垂直居中
- 【当前】 偏上约 2px (见截图)
- 【期望】 与 send 按钮中线对齐
- 【范围】 只改 composer-toolbar 内 flex 对齐, 不动间距 4px 设定

像素调整一次说清：当前值、目标值、参考截图。避免「没看到效果 → 再调 4px → 再调 2px」循环。

D. 原理 / 设计

- 【问题】 上下文超阈值后的处理流程
- 【输出】 1. 流程图 2. 现有几种压缩及代码位置 3. 与 Claude Code 差异
- 【下一步】 先别改代码，我确认方案后再实现

你做过最好的例子就是上下文压缩那条线：先分析 → 对比 → 实现 → Settings 开关。

截图 / 附件用法

附件类型	建议附带
截图	标注：红框 = 要改的；箭头 = 期望位置；「现在 / 想要」各一句
录屏	说明哪几秒是 bug（如 0:03 列表跳动）
参考文件	说明用哪一段（如 auth 文件的菜单文案，不是整文件重写）
终端日志	用 @terminals/xxx.txt:行号 或贴关键 10 行

跟进消息怎么写

「没看到效果」可以改成：

重启了 npm run dev，硬刷新后仍无效。
当前：复制按钮仍在左侧
已试：清缓存 / 换 session
请检查 chat.css 里 .message-copy 是否被覆盖

「修复好了吗」可以改成：

上一条：Agent 设置页窗口缩放控件不跟随
请确认是否已改 settings 的 flex/grid，并说明如何验证（拖窗口到 xx 宽度）

范围控制（建议坚持）

这些提问 Agent 完成度很高：

- 「只要处理 Agent/Context 设置页，其他地方不要处理」
- 「不要随便改动其他代码，需要提前告诉我（卡帕奇合约）」
- 「/clear 不清 UI，只清 LLM 上下文」

每条需求都加一句「不要动：xxx」，能减少无关 diff。

三、对话组织建议

- 同一模块的 UI（如 composer-toolbar）→ 一个对话里批量改，避免 12 个对话各改 1px
- Bug 与 UI → 分开
- 「继续」→ 带上「上一步做到哪、卡在哪」
- 重复需求 → 新对话里写「延续 Composer 工具栏改造，还差 context 弹窗对齐」

四、针对你习惯的「快捷改进清单」

1. 超过 3 条需求 → 编号 + 标 P0/P1，或拆对话
2. 每条 UI 需求 → 类名 + 截图 + 验收一句
3. 说「像 Cursor」→ 指明：布局 / 颜色 / 交互 / 字体 中的哪一项
4. 改完说「没看到效果」→ 补：怎么跑的、当前实际表现
5. 像素调整 → 尽量一次写「从 A 到 B」，或截图像素标尺
6. 大改前先 Plan → 「先出方案，我确认再写代码」（上下文压缩那条就很好）
7. 固定验收口令 → 「改完后告诉我改了哪些文件 + 如何验证」

五、总体评价

你是在 **产品级打磨** 一个 Electron Agent 客户端：从迁移、Agent 能力到 UI 像素级对齐 Cursor。提问 **技术定位准**（类名、路径、对标产品），主要损耗在 **批次过大、跟进信息过少、UI 微调轮次多**。

若只改一处习惯，建议：

大需求拆优先级；小 UI 一次说清「元素 + 现状 + 期望 + 验收」；跟进时永远带上「我这边看到什么」。